

Week 6 demo — fallback captures

Bike-sharing “rent a bike” sequence diagram

Captured with Claude Code, Sonnet 5 (low effort), September 2026

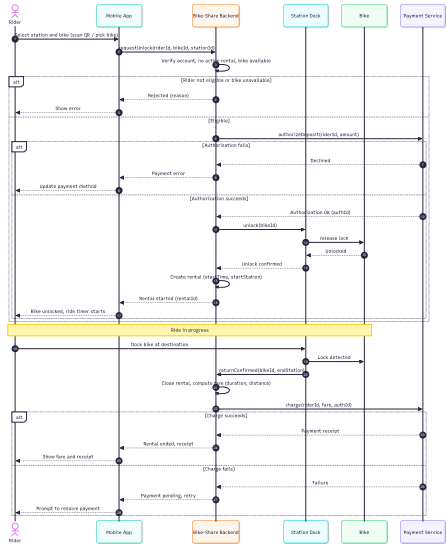
When to use this deck

Switch here if the live AI fails. These are real answers to the demo's own prompts; walk the same defect catalogue against them.

Prompt #1

“Generate a UML sequence diagram (as Mermaid) for renting a bike in the city bike-sharing app: a rider unlocks a bike at a station and is charged by app.”

AI's answer — the sequence



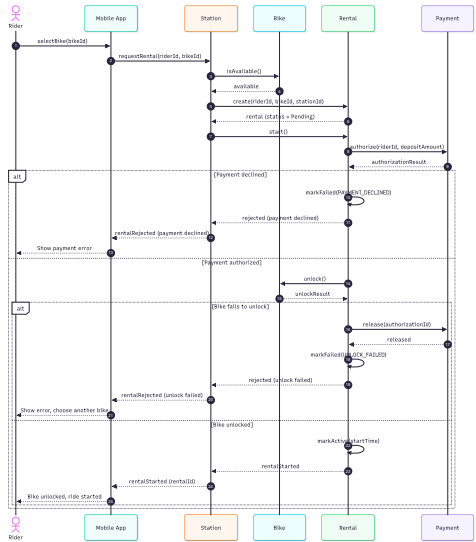
AI's answer — what it said

The diagram covers the full flow: eligibility check, deposit authorization, unlock via the station, then fare calculation and charging when the bike is docked. It also includes failure paths for ineligible riders, declined authorization, and failed final charges.

Prompt #2 (same session)

“Revise the sequence. Model only renting — stop once the ride has started. Replace the Backend’s self-calls with the domain objects from our class diagram: show which object creates the Rental and which one handles the Payment. Authorise payment before the unlock, and add alt fragments for payment declined and for the bike failing to unlock. Every call that returns a result must show its dashed return.”

AI's answer — the revised sequence



AI's answer — its change list

I couldn't see your class diagram, so I assumed these domain objects: `Station`, `Bike`, `Rental` and `Payment`. `Station` creates the `Rental`, and the `Rental` owns and calls its `Payment`. Rename them if your diagram differs.

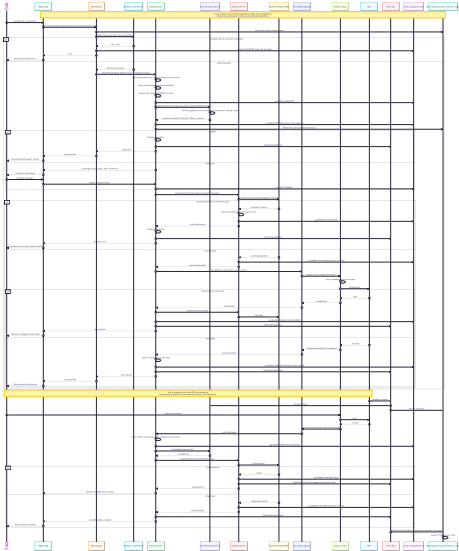
[...]

- ▶ **Creation:** *`Station` creates the `Rental`. `Rental` then runs the payment and unlock steps in order.*
- ▶ **Payment:** *the `Rental` calls `Payment.authorize` before `Bike.unlock`. If the bike then fails to unlock, the `Rental` calls `Payment.release` to void the hold.*
- ▶ **Returns:** *every call that yields a result has a dashed return, including `create`, `authorize`, `unlock` and `release`.*
- ▶ **Self-calls:** *the only ones left are `Rental` updating its own status (`markFailed`, `markActive`), which is the object's own behaviour, not the Backend's.*
- ▶ **Scope:** *the diagram ends once the ride has started, so docking and the final charge are out.*

Reserve prompt (fresh session, after prompt #1)

"Make this enterprise-grade with full observability, fraud checks, and audit logging."

AI's answer — the enterprise sequence



Zoom — the fraud decision

```
alt DENY
    Rental->>Rental: Release reservation
    Rental->>Bus: Publish RentalDenied
    Rental-->>GW: 403 Denied
    GW-->>App: Unlock denied
    App-->>Rider: Show denial and support contact
else STEP_UP
    Rental-->>App: Challenge required (3DS / OTP / biometric)
    App-->>Rider: Prompt for verification
    Rider->>App: Complete challenge
    App->>Rental: Submit challenge result
    Rental->>Audit: Log STEP_UP_RESULT
end

%% ----- Payment pre-authorization -----
Rental->>Pay: authorize(riderId, depositAmount, idempotencyKey)
```

Zoom — messages no requirement asked for

```
    GW->>Obs: Start trace, record request metric
[...]
```

 Rental->>Audit: Log FRAUD_DECISION (score, rules triggered)

 Rental->>Obs: Metric fraud_decision_total{decision}

```
[...]
```

 Bike-)Bus: Telemetry stream

 Bus-)Fraud: Stream analysis

```
[...]
```

 Obs->>Obs: Evaluate SLO and alert rules

AI's answer — what it said (reserve)

What was added:

- ▶ **Observability:** *trace and correlation IDs on every hop, metrics (fraud decisions, latency, success rates), structured logs, telemetry streaming, and SLO alerting.*
- ▶ **Fraud checks:** *a pre-rental risk assessment with allow, step-up, or deny outcomes. It also covers in-ride anomaly monitoring and a post-ride review.*
- ▶ **Audit logging:** *an append-only audit log records each security and financial event, [...]*
- ▶ **Resilience and security:** *[...]*

The diagram is large. I can split it into separate diagrams for unlock, ride monitoring, and return and payment if you'd like.